

ASP.NET Core Custom Token Provider

- **DataProtectorTokenProvider** class can generate different types of tokens
- Token life span is controlled by **DataProtectionTokenProviderOptions** class
- The default token life span is **1 day**

```
public void ConfigureServices(IServiceCollection services)
{
    // Changes token life span of all token types
    services.Configure<DataProtectionTokenProviderOptions>(o =>
        o.TokenLifespan = TimeSpan.FromHours(5));
}
```

DataProtectorTokenProvider прави промени генерално върху всички видове токени – за пароли, имейли и прочие. Ако искаме да персонализираме конкретно за даден специфичен вид, например валидността на токена на имейлите да бъде 3 дена, трябва да персонализираме двата класа:

DataProtectorTokenProvider + DataProtectionTokenProviderOptions, Като помним че във втория Options се намира продължителността за TimeSpan. Така че може да използваме вградените му възможности и унаследявания наготово.

`services.Configure<DataProtectionTokenProviderOptions>` ☐ Go To Definition

```
using System;

namespace Microsoft.AspNetCore.Identity
{
    public class DataProtectionTokenProviderOptions
    {
        public DataProtectionTokenProviderOptions();

        public string Name { get; set; }
        public TimeSpan TokenLifespan { get; set; }
    }
}
```

Security folder -> Add -> Class ->

```
namespace EmployeeManagement.Security
{
    0 references
    public class CustomEmailConfirmationTokenProviderOptions : DataProtectionTokenProviderOptions
    {
    }
}
```

```
using Microsoft.AspNetCore.Identity;
```

```
namespace EmployeeManagement.Security
{
    public class CustomEmailConfirmationTokenProviderOptions :
DataProtectionTokenProviderOptions
    {
    }
}
```

Step 1 : Create Custom Options & Provider

```
public class CustomEmailConfirmationTokenProviderOptions
    : DataProtectionTokenProviderOptions
{
}
```

We pass our custom parameter instead of
DataProtectionTokenProviderOptions
because of inheritance relationship :

```
public class CustomEmailConfirmationTokenProvider<TUser>
    : DataProtectorTokenProvider<TUser> where TUser : class
{
    public CustomEmailConfirmationTokenProvider(IDataProtectionProvider dataProtectionProvider,
        IOptions<CustomEmailConfirmationTokenProviderOptions> options)
        : base(dataProtectionProvider, options)
    {
    }
}
```

We pass <TUser>

github.com/aspnet/identity/blob/release/2.2/src/identity/DataProtectionTokenProvider.cs

```
namespace Microsoft.AspNetCore.Identity
{
    /// <summary>
    /// Provides protection and validation of identity tokens.
    /// </summary>
    /// <typeparam name="TUser">The type used to represent a user.</typeparam> where TUser is just a class
    public class DataProtectorTokenProvider<TUser> : IUserTwoFactorTokenProvider<TUser> where TUser : class
    {
        public DataProtectorTokenProvider(IDataProtectionProvider dataProtectionProvider, IOptions<DataProtectionTokenProviderOptions>
        {
            constructor ( parameter 1 ; our own custom parameter 2 )
        }
    }
}
```

Let's add our custom class also to the security folder (without Options):

```

CustomEmailC...nProvider.cs* X CustomEmailC...derOptions.cs DataProtecti...om metac
agement EmployeeManagement.Security.CustomEmailConfigur CustomEmailConfirmationTokenProvid
using Microsoft.AspNetCore.DataProtection;
using Microsoft.Extensions.Options;

namespace EmployeeManagement.Security
{
    1 reference
    public class CustomEmailConfirmationTokenProvider<TUser>
        where TUser : class
    {
        0 references
        public CustomEmailConfirmationTokenProvider(IDataProtectionProvider dataProtectionProvider,
            IOptions<CustomEmailConfirmationTokenProviderOptions> options)
            : base(dataProtectionProvider, options)
        { }
    }
}

```

base class DataProtectorTokenProvider in core 3.0 has addition parameter ILogger, the following code is working:

```

public CustomEmailConfirmationTokenProvider(IDataProtectionProvider dataProtectionProvider,
    IOptions<CustomEmailConfirmationTokenProviderOptions> options,
    ILogger<CustomEmailConfirmationTokenProvider<TUser>> logger)
    : base(dataProtectionProvider, options, logger)
{ }

```

```

using Microsoft.AspNetCore.DataProtection;
using Microsoft.AspNetCore.Identity;
using Microsoft.Extensions.Logging;
using Microsoft.Extensions.Options;
using NLog;

```

```

namespace EmployeeManagement.Security
{
    public class CustomEmailConfirmationTokenProvider<TUser>
        : DataProtectorTokenProvider<TUser> where TUser : class
    {
        public CustomEmailConfirmationTokenProvider(IDataProtectionProvider
dataProtectionProvider,
            IOptions<CustomEmailConfirmationTokenProviderOptions> options,
            ILogger<CustomEmailConfirmationTokenProvider<TUser>> logger)
            : base(dataProtectionProvider, options, logger)
        { }
    }
}

```

}

Step 2 : Register Custom Token Provider

```
public void ConfigureServices(IServiceCollection services)
{
    // Rest of the code

    services.AddIdentity<ApplicationUser, IdentityRole>(options =>
    {
        options.Tokens.EmailConfirmationTokenProvider = "CustomEmailConfirmation";
    })
    .AddEntityFrameworkStores<AppDbContext>()
    .AddDefaultTokenProviders()
    .AddTokenProvider<CustomEmailConfirmationTokenProvider<ApplicationUser>>("CustomEmailConfirmation");

    // Rest of the code
}
```

```
public void ConfigureServices(IServiceCollection services)
{
    services.AddDbContextPool<AppDbContext>(
        options => options.UseSqlServer(_config.GetConnectionString("EmployeeDBConnection")));

    services.AddIdentity<ApplicationUser, IdentityRole>(options =>
    {
        options.Password.RequiredLength = 10;
        options.Password.RequiredUniqueChars = 3;

        options.SignIn.RequireConfirmedEmail = true;
        options.Tokens.EmailConfirmationTokenProvider = "CustomEmailConfirmation";
    })
    .AddEntityFrameworkStores<AppDbContext>()
    .AddDefaultTokenProviders()
    .AddTokenProvider<CustomEmailConfigurationTokenProvider<ApplicationUser>>("CustomEmailConfirmation");
```

```
options.Tokens.EmailConfirmationTokenProvider = "CustomEmailConfirmation";
    }).AddEntityFrameworkStores<AppDbContext>()
    .AddDefaultTokenProviders()
```

```
.AddTokenProvider<CustomEmailConfigurationTokenProvider<ApplicationUser>>("CustomEmailConfirmation");
```

```
options.Tokens.EmailConfirmationTokenProvider = "CustomEmailConfirmation";
}).AddEntityFrameworkStores<AppDbContext>()
.AddDefaultTokenProviders()
.AddTokenProvider<CustomEmailConfigurationTokenProvider<ApplicationUser>>("CustomEmailConfirmation");

//services.AddAuthorization(options =>
//options.AddPolicy("Role",
//policy => policy.RequireRole("Admin")));

services.Configure<DataProtectionTokenProviderOptions>(o =>
    o.TokenLifespan = TimeSpan.FromHours(5));
services.Configure<CustomEmailConfirmationTokenProviderOptions>(o =>
    o.TokenLifespan = TimeSpan.FromDays(3));

services.AddMvc(options =>
```

```
services.Configure<CustomEmailConfirmationTokenProviderOptions>(o =>
    o.TokenLifespan = TimeSpan.FromDays(3));
```


Step 3 : Change email conf. token lifespan

```
public void ConfigureServices(IServiceCollection services)
{
    // Changes token lifespan of all token types
    services.Configure<DataProtectionTokenProviderOptions>(o =>
        o.TokenLifespan = TimeSpan.FromHours(5)); all tokens

    // Changes token lifespan of just the Email Confirmation Token type
    services.Configure<CustomEmailConfirmationTokenProviderOptions>(o =>
        o.TokenLifespan = TimeSpan.FromDays(3)); only email tokens
}
```